reduce, destructuring y spread/rest

Repaso con productos

```
const productos = [
  { nombre: "Teclado", precio: 30, stock: 4 },
  { nombre: "Ratón", precio: 15, stock: 0 },
  { nombre: "Monitor", precio: 180, stock: 3 },
  { nombre: "Webcam", precio: 55, stock: 2 }
];
```

Antes de programar, identificamos qué resultado necesitamos.

Cinco necesidades del catálogo

1. Obtener los productos con stock — **Seleccionar elementos → filter()**
2. Obtener solamente sus nombres — **Transformar cada elemento → map()**
3. Localizar el Monitor — **Buscar un elemento → find()**
4. Saber si alguno cuesta más de 100 € — **Comprobar si alguno cumple una condición → some()**
5. Comprobar si todos cuestan más de 10 € — **Comprobar todos → every()**

¿Qué operación resuelve cada necesidad?

Qué devuelve cada método

map()	Un array con los resultados de transformar
filter()	Un array con los elementos seleccionados
find()	Un elemento o undefined
some()	Un booleano: alguno cumple
every()	Un booleano: todos cumplen

La forma del resultado ayuda a elegir el método.

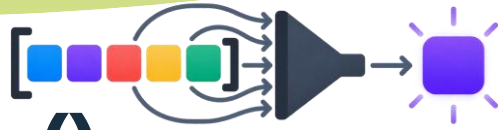
Un acumulador con for...of

```
const precios = [10, 20, 5, 15];
let total = 0;

for (const precio of precios) {
  total = total + precio;
}

console.log(total); // 50
```

¿Qué valor conserva total entre una vuelta y la siguiente?

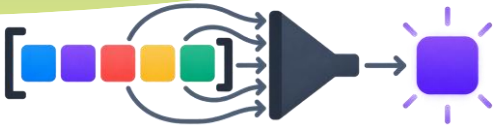


Hagámoslo con un método de Array reduce().

```
const precios = [10, 20, 5, 15];

const total = precios.reduce(
  (acumulador, precio) => {
    return acumulador + precio;
  },
  0
);
```

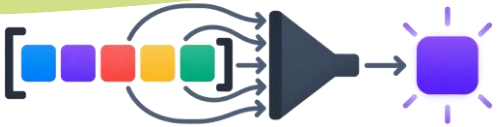
- Sintaxis arr.reduce(callback(acumulador, valorActual)[, valorInicial])
- Si no se indica valorInicial, toma **el primer elemento del array como valor inicial del acumulador** y empieza a recorrer desde el segundo elemento.
- El callback devuelve el nuevo acumulador. El 0 es su valor inicial, este valor es opcional.



El acumulador paso a paso

Vuelta	Acumulador	Precio	Devuelve
1	0	10	10
2	10	20	30
3	30	5	35
4	35	15	50

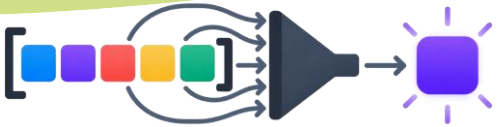
Resultado final: 50. Cada devolución se usa en la siguiente vuelta.



Acumular una propiedad

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 },
  { nombre: "Monitor", precio: 180 }
];
const total = productos.reduce(
  (acumulador, producto) => acumulador + producto.precio,
  0
); // 225
```

Cada elemento es un objeto. La cantidad que sumamos es su precio.

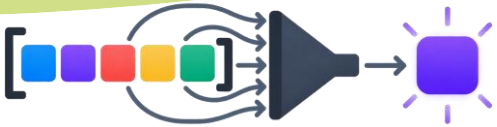


Ejercicio guiado: nota media

```
const alumnos = [
  { nombre: "Ana", nota: 7 },
  { nombre: "Luis", nota: 5 },
  { nombre: "Eva", nota: 9 },
  { nombre: "Pablo", nota: 3 }
];

// Calcula la nota media del grupo.
```

¿Qué cantidad necesitas obtener antes de calcular la media?



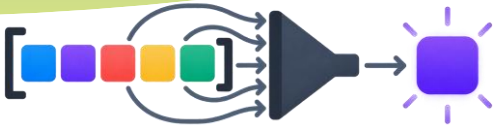
Corrección: suma y media

```
const suma = alumnos.reduce(
  (total, alumno) => total + alumno.nota,
  0
);

const media = alumnos.length > 0
  ? suma / alumnos.length
  : null;

console.log(media); // 6
```

Un grupo vacío no tiene media. En este ejemplo devolvemos null.



reduce también puede elegir un máximo

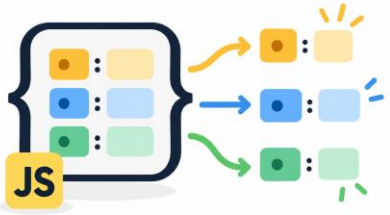
```
const precios = [30, 15, 180, 55, 70];

const mayor = precios.reduce(
  (mayor, precio) => precio > mayor ? precio : mayor
);

console.log(mayor); // 180
```

Sin valor inicial, empieza con el primer elemento. Un array vacío falla.

Destructuring de objetos




```
const alumno = { nombre: "Ana", edad: 20, nota: 8 };

const { nombre, nota } = alumno;

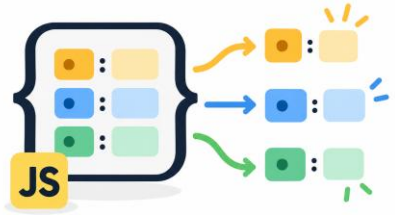
const {nombre: nombreAlumno, nota: notaAlumno} = alumno;

console.log(nombre); // Ana
console.log(nota);   // 8

console.log(nombreAlumno); // Ana
console.log(notaAlumno);   // 8
```

Extraemos propiedades por su nombre y las guardamos en variables. Es posible cambiar el nombre de variable que alamcena la desetructuración.

Destructuring de objetos




```
const alumno = { nombre: "Ana", edad: 20, nota: 8 };

const { nombre, nota } = alumno;

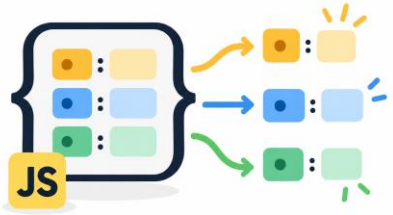
console.log(nombre); // Ana

alumno.nombre = “Pepe”;

console.log(nombre, alumno.nombre); // Ana Pepe
```

Si un objeto cambia una propiedad y es un valor primitivo, solo se cambia en el objeto y no en la variable desestructurada.

Destructuring de objetos




```
const alumno = { nombreCompleto: {nombre: “Ana"}, edad: 20, nota: 8 };

const { nombreCompleto, nota } = alumno;

console.log(nombreCompleto.nombre); // Ana

alumno.nombreCompleto.nombre = “Pepe”;

console.log(nombreCompleto.nombre, alumno.nombreCompleto.nombre); // Pepe Pepe
```

Pero si la propiedad es un objeto y se cambia un valor de ese objeto, al desestructurar el objeto principal, como la variable desestructurada almacena la dirección del objeto. Cambia los valores almacenados cuando se cambian sus valores.

Desestructuración de objetos en JavaScript

Extraer propiedades de un objeto y guardarlas en variables.

Objeto

```
const persona = {
  nombre: "Ana",
  edad: 25,
  ciudad: "Sevilla"
};
```

Desestructuración

```
const { nombre, edad }
    = persona;
```

Variable

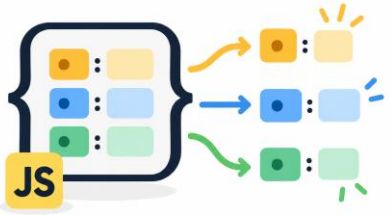
nombre → "Ana"

Variable

edad → 25

Las **claves del objeto** se extraen en **variables independientes.**

Destructuring de arrays



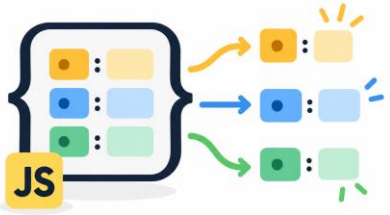

```
const colores = ["rojo", "verde", "azul"];

const [primero, segundo] = colores;

console.log(primero); // rojo
console.log(segundo); // verde
```

En objetos importa el nombre de la propiedad. En arrays, la posición.

Práctica breve: datos de un producto

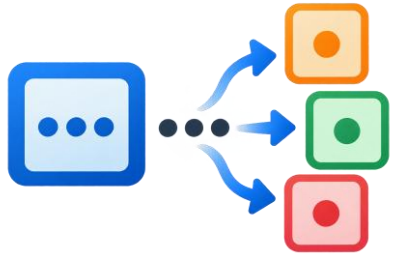



```
const producto = {
  id: 4,
  nombre: "Webcam",
  precio: 55,
  stock: 2
};

// Extrae nombre y precio en variables.
```

Muestra en la consola: Webcam cuesta 55 €

Spread expande los elementos



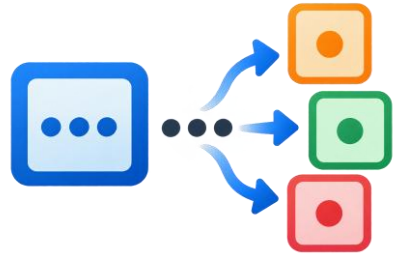
```
const numeros = [10, 20, 30];

console.log(numeros);    // [10, 20, 30]

console.log(...numeros); // 10 20 30
```

En la segunda llamada, cada elemento se convierte en un argumento.

Nuevos arrays con spread

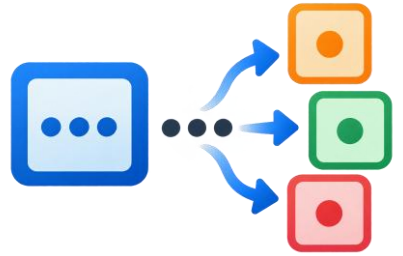


```
const numeros1 = [10, 20, 30];
const numeros2 = [...numeros1, 40, 50];
// [10, 20, 30, 40, 50]

const grupo1 = ["Ana", "Luis"];
const grupo2 = ["Marta", "Pablo"];
const alumnos = [...grupo1, ...grupo2];
// ["Ana", "Luis", "Marta", "Pablo"]
```

El array exterior es nuevo. Los arrays originales conservan sus elementos.

Un nuevo objeto con spread

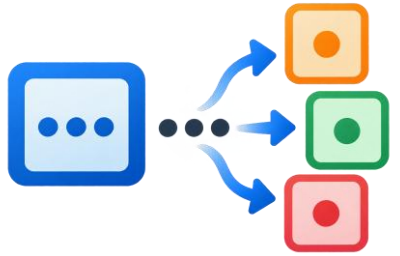


```
const jugador = {
  nombre: "Laura", nivel: 4, puntos: 850
};
const copia = { ...jugador };

console.log(jugador === copia); // false
copia.puntos = 1000;
console.log(jugador.puntos);    // 850
console.log(copia.puntos);      // 1000
```

Los objetos exteriores son diferentes. puntos contiene un número.

Una versión modificada del jugador

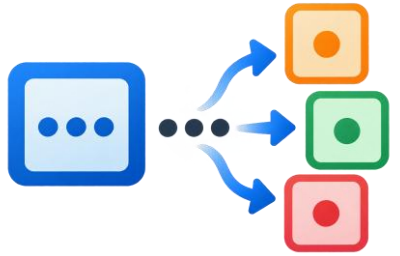


```
const jugadorActualizado = {
  ...jugador,
  nivel: 5,
  puntos: 1000
};

// { nombre: "Laura", nivel: 5, puntos: 1000 }
// jugador conserva nivel: 4 y puntos: 850
```

Primero copiamos las propiedades. Después sustituimos las que cambian.

El orden de las propiedades importa



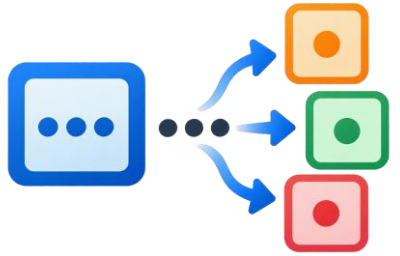
```
const a = { nombre: "Ana", edad: 20 };

const b = { ...a, edad: 25 };
const c = { edad: 25, ...a };

console.log(b.edad); // 25
console.log(c.edad); // 20
```

Si una propiedad aparece varias veces, prevalece el último valor.

Copia superficial: valores primitivos



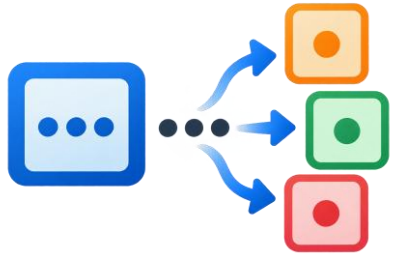
```
const producto = { nombre: "Monitor", precio: 180 };
const copia = { ...producto };

producto.precio = 200;

console.log(producto.precio); // 200
console.log(copia.precio);    // 180
```

Cambiar esta propiedad numérica no cambia la propiedad de la copia.

Copia superficial: objetos anidados



```
const alumno = {
  nombre: "Ana",
  direccion: { ciudad: "Sevilla" }
};
const copia = { ...alumno };

console.log(alumno === copia);                    // false
console.log(alumno.direccion === copia.direccion); // true
alumno.direccion.ciudad = "Cádiz";
console.log(copia.direccion.ciudad);              // Cádiz
```

direccion sigue apuntando al mismo objeto en ambos casos.

Rest agrupa argumentos



```
function mostrar(...numeros) {
  console.log(numeros);
}

mostrar(10, 20, 30);
// Dentro de la función: [10, 20, 30]
```

En los parámetros, ... recoge los argumentos en un array.

Rest y reduce trabajan juntos



```
function sumar(...numeros) {
  return numeros.reduce(
    (total, numero) => total + numero,
    0
  );
}

sumar(10, 20);     // 30
sumar(10, 20, 30); // 60
sumar();           // 0
```

El valor inicial 0 también permite sumar un array vacío.

Rest en la desestructuración

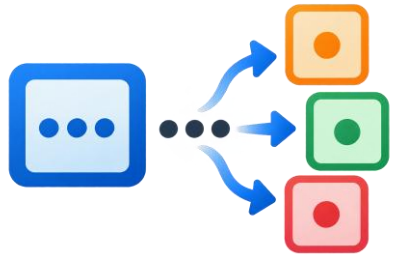


```
const numeros = [10, 20, 30, 40, 50];
const [primero, segundo, ...resto] = numeros;
// primero: 10, segundo: 20, resto: [30, 40, 50]

const alumno = { nombre: "Ana", edad: 20, nota: 8 };
const { nombre, ...otrosDatos } = alumno;
// nombre: "Ana"
// otrosDatos: { edad: 20, nota: 8 }
```

Rest recoge los elementos o propiedades que quedan sin extraer.

¿Spread o rest?





```
const numeros = [10, 20, 30, 40, 50];

const copia = [...numeros];

function sumar(...numeros) { /* ... */ }

const nuevo = { ...producto, precio: 50 };

const [primero, ...resto] = numeros;
```

¿Expande una estructura existente o recoge varios valores?

Ejercicio integrador: catálogo



```
const productos = [
  { id: 1, nombre: "Teclado", precio: 30, stock: 4 },
  { id: 2, nombre: "Ratón", precio: 15, stock: 0 },
  { id: 3, nombre: "Monitor", precio: 180, stock: 3 },
  { id: 4, nombre: "Webcam", precio: 55, stock: 2 }
];
```

20 minutos. Funciones con parámetros y retorno, sin modificar los datos.

Tres funciones para el catálogo



obtenerNombresDisponibles(productos)
Devuelve los nombres de los productos con stock.

calcularValorStock(productos)
Devuelve la suma de precio × stock de los disponibles.

actualizarPrecio(producto, nuevoPrecio)
Devuelve un nuevo producto con el precio indicado.

La elección de las herramientas forma parte del ejercicio.

Comprobación del ejercicio



Nombres esperados
["Teclado", "Monitor", "Webcam"]

Valor del stock disponible
770

Actualización de precio
El resultado cambia. El producto original se conserva.

Prueba también arrays vacíos y entradas incorrectas razonables.

Antes de la EV1

¿Qué datos recibe la función y qué debe devolver?

¿Qué operación necesita el problema?

¿Qué valor devuelve el callback en cada vuelta?

¿Estoy modificando los datos originales?

¿Qué ocurre con entradas vacías o incorrectas?

Una solución debe funcionar y poder explicarse.